Complete NLP Preprocessing Pipeline and Sentence Representation

When working with text in Natural Language Processing (NLP), we transform raw text into numerical representations through a series of processing steps. These steps convert the original text into tokens, map them to dense vectors (embeddings), and then combine these token embeddings into sentence-level representations. In this note, we outline the complete process.

1 Preprocessing (No Mathematical Operations Yet)

1.1 Text Normalization

Purpose: Clean and standardize raw text.

Techniques:

- **Lowercasing:** Convert all characters to lowercase.
- **Punctuation Removal/Standardization:** Remove or standardize punctuation.
- **Whitespace Standardization:** Replace multiple spaces or newlines with a single space.
- **Accent and Diacritic Removal:** Convert accented characters to their unaccented forms.
- **Expanding Contractions:** Replace contractions (e.g., "don't" to "do not").
- **Spelling Correction:** Correct common typos and misspellings.

Outcome: A cleaner, more uniform text ready for further processing.

1.2 Stemming and Lemmatization

Purpose: Reduce words to their base forms to reduce variability.

Stemming:

- Uses heuristic rules (e.g., Porter, Snowball) to strip affixes.
- Example: "running", "runs", and "ran" may all be reduced to "run" (or a common stem form).

Lemmatization:

- Uses linguistic rules and dictionaries (often with POS tagging) to convert words to their dictionary (lemma) forms.
- Example: "running" → "run", "better" (as adjective) → "good".

Decision: Depending on the application, you might use stemming, lemmatization, or both. **Result:** New sentences (or token sequences) in which words have been replaced by their stemmed or lemmatized forms.

In Natural Language Processing (NLP), preprocessing steps such as stemming and lemmatization are used to reduce words to their base forms. This simplifies the vocabulary by grouping together different inflected forms (e.g., "run", "runs", "ran", and "running" might all be reduced to "run"). However, these processes also remove morphological details that are important for learning the grammatical structure of a language.

- **Stemming** is a heuristic method that crudely chops off suffixes and prefixes. It can produce non-dictionary "stems" (e.g., "running" might become "run" or even "runn") and may conflate words that have different meanings.
- **Lemmatization** uses linguistic rules and often part-of-speech (POS) tags to convert words to their canonical forms (lemmas). For example, it can map "better" to "good" (when used as an adjective) and "running" to "run." However, by converting words to their lemmas, the system loses information about the original inflection (e.g., whether a verb is in the past tense or present continuous).
- The primary goal is to capture the overall meaning or topic of the text rather than the precise morphological form.
- Reducing word forms via stemming/lemmatization can be beneficial by reducing vocabulary size and sparsity, allowing the model to focus on core semantic content.
- The model must produce grammatically correct, naturally inflected words.
- If the training data has been heavily stemmed or lemmatized, the model might only learn the base forms (e.g., "catch") and not the correct inflections (e.g., "caught").
- For generative tasks, preserving the original forms (or using subword tokenization methods that retain morphological structure) is crucial so that the model can learn and generate proper grammar.
- **Subword Tokenization:** Techniques such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece break words into subword units. These methods allow models to learn morphology and inflection patterns because even if a word is split into parts, the model can learn how those parts combine to form various word forms. Importantly, subword tokenization does not perform stemming or lemmatization.

- **Contextual Models:** Modern models (e.g., BERT, GPT) are typically trained on raw or minimally processed text. They learn both semantic and grammatical structure from large-scale data, without the need for heavy preprocessing like stemming or lemmatization.
- **Task-Specific Pipelines:** In some cases, a two-stage process may be used: one pipeline for understanding (using stemmed/lemmatized text) and another for generation (using full, raw text), or a post-processing step may convert lemmas back into the proper inflected forms based on context.
- **When to Use Stemming/Lemmatization:** They are beneficial when the task focuses on the general meaning of the text (e.g., topic classification or sentiment analysis) and where reducing the vocabulary size is advantageous.
- **When to Avoid or Modify Them:** For tasks that require precise grammatical output or natural language generation, it is preferable to work with the original forms or use subword-based methods.
- **Learning Grammar:** Models that need to generate text must learn from data that includes the full range of inflections. If a model has never seen "caught" because all training instances were lemmatized to "catch," it won't learn the grammatical rules required to produce "caught" in the appropriate context.

In Summary, Stemming and Lemmatization reduce vocabulary complexity and group similar word forms, which benefits understanding tasks. However, they remove important morphological information needed for learning and generating correct grammatical forms. For **generative tasks**, it is crucial to preserve or recover inflectional details—this can be achieved by using raw text, subword tokenization, or specialized morphological generation components. **Modern NLP systems** often rely on end-to-end models that learn grammar and semantics directly from raw or subword-tokenized text, thereby bypassing the need for heavy stemming/lemmatization.

1.3 Tokenization

Purpose: Split the processed text into tokens.

Techniques:

- **Word-Level Tokenization:** Split based on whitespace and punctuation.
- **Subword Tokenization:** Use algorithms such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece.
- **Character-Level Tokenization:** Split the text into individual characters.

Outcome: A sequence of tokens representing the sentence.

1.4 Dictionary (Vocabulary) Construction

Purpose: Build a mapping from unique tokens to unique indices.

Outcome: A vocabulary (dictionary) of tokens used to convert text into numerical indices.

Note: Up to this point, the steps are largely text processing and mapping, with no mathematical vector operations performed yet.

2 Converting Tokens to Numerical Representations

2.1 One-Hot Encoding

Process: Each token is first represented as a one-hot vector, which is a high-dimensional sparse vector with a single 1 at the token's index and 0's elsewhere.

2.2 Embedding Layer (Token Embeddings)

Purpose: Convert one-hot vectors into dense, low-dimensional representations (embeddings) that capture semantic and syntactic information.

Techniques:

- **Random Initialization:** Embeddings are initialized randomly and learned during training.
- **Pre-Trained Embeddings:** Use embeddings such as Word2Vec, GloVe, or fastText, pre-trained on large corpora.
- **Subword/Character-Based Embeddings:** For subword or character tokenization, learn embeddings that capture morphological details.

Key Point: At this stage, we focus solely on obtaining dense embeddings for individual tokens. We do not yet aggregate them into sentence-level representations.

3 Representing Sentences

Once each token is embedded as a dense vector, these token embeddings must be aggregated into a fixed-size sentence representation. The aggregation method depends on the network architecture:

3.1 Recurrent Neural Networks (RNNs) / LSTMs / GRUs

Process:

- Feed the sequence of token embeddings into an RNN.
- The network maintains a hidden state that is updated at each time step.

Sentence Representation: The final hidden state (or a combination via an attention mechanism) is used as the sentence embedding.

3.2 Convolutional Neural Networks (CNNs)

Process:

- Apply convolutional filters over fixed-size windows of token embeddings to capture local n-gram features.
- Use pooling (max or average) to aggregate features.

Sentence Representation: The pooled vector provides a fixed-size representation of the sentence.

3.3 Simple Aggregation Methods

Process:

- Combine token embeddings by simple operations such as averaging or summing.

Sentence Representation: The aggregated vector represents the sentence.

3.4 Attention Mechanisms

Process:

- Compute weights for each token embedding based on its importance.
- Take a weighted sum of the token embeddings.

Sentence Representation: The weighted sum emphasizes the most relevant tokens, forming the sentence vector.

Note: Even though Transformer networks use self-attention to produce sentence representations, the above methods describe how sentence embeddings are constructed in non-Transformer architectures.

4 Recap

1. **Preprocessing:**
    - Normalize text (cleaning, lowercasing, punctuation removal, etc.).
    - Apply stemming and/or lemmatization to reduce words to their base forms.
    - Tokenize the processed text.
    - Build a vocabulary (dictionary) mapping tokens to indices.
2. **Token-to-Vector Conversion:**
    - Represent each token as a one-hot vector.
    - Use an embedding layer to convert one-hot vectors into dense token embeddings.
3. **Sentence Representation:**
    - Aggregate token embeddings into a fixed-size sentence vector using RNNs/LSTMs, CNNs, simple pooling, or attention mechanisms.

This pipeline converts raw text into numerical representations that are amenable to machine learning models. The initial steps (normalization, stemming/lemmatization, and tokenization) prepare the text without any mathematical operations. Once tokens are obtained and mapped to indices via a dictionary, one-hot encoding and embedding create dense vector representations. Finally, sentence-level representations are built from these token embeddings, with the choice of aggregation method depending on the architecture used.